COMP.8157 ADVANCED DATABASE TOPICS | GROUP PROJECT DELIVERABLE D.4.2

Analysis and Design Document

A Change Stream Ledger for MongoDB: Tamper Evidence Without a Ledger Engine

Course	COMP.8157 Advanced Database Topics, Fall 2026, University of Windsor. Instructor: Dr. Andreas S. Maniatis
Team	Olumide Ogunjebi (ogunjeb@uwindsor.ca), Rohith Nelluri (nellurir@uwindsor.ca), Yadu Pratap (Singh8a4@uwindsor.ca)
Document	30 October 2026. Status: submitted for review.
Code repository	GitHub: https://github.com/OlumideOgunjebi/mongoledger
Project management	Hive: https://app.hive.com/workspace/vdotT67a4S5s8bEPt?projectId=SnqAeBtxRHMFEjjam

Contents

1. Introduction and Context 3
1.1 Purpose and audience 3
1.2 Problem being solved 3
1.3 Scope 3
1.4 Link to the requirements 3
2. Goals and Nongoals 3
3. System Architecture 4
3.1 Logical architecture 4
3.2 Physical architecture 5
4. Data Model 6
4.1 Entity relationship model 6
4.2 Hashes and canonical encoding 6
4.3 Block structure 7
4.4 Data flow 8
5. Component Design 8
5.1 Merkle root and inclusion proof (merkle.py) 8
5.2 Design S write and fold (design_s.py) 9
5.3 Design A service (design_a.py) 9
5.4 Verifier (verifier.py) 9
5.5 Search and proofs (search.py) 9
6. Interface Design 10
6.1 Command line interface 10
6.2 Screen layouts 11
6.3 Python API 11
6.4 Protocols and file formats 11
7. Constraints and Security 12
7.1 Hardware and environment limits 12
7.2 Performance bottlenecks 12
7.3 Security considerations 12
8. Design Rationale 12
9. Revision History 13
References 14

1. Introduction and Context

1.1 Purpose and audience

This document explains how the system is built and why. It is written for the team, for the graders who will run and read the code, and for anyone who later extends the ledger. It refines the requirements in the User Requirements and Analysis Document (D.4.1) into components, data structures, algorithms and interfaces, and every design element below names the requirements it serves.

1.2 Problem being solved

MongoDB has no ledger. An insider with administrator rights can change a protected document, delete history or restore an old backup, and the database keeps no evidence that an auditor could check later. The system adds that evidence to an unmodified MongoDB 8.0 replica set. Every change to a protected collection becomes a ledger entry; entries are sealed into hash chained Merkle blocks; the hash of every block is published outside the database; and a verifier compares the ledger, the published digests and the live data. Two capture designs are offered: **Design S** writes entries inside the application's own transaction, and **Design A** derives them afterwards from MongoDB change streams.

1.3 Scope

In scope are the ledger library and service, the verifier and search tools, the SQL Server 2022 Ledger baseline, three workloads (SmallBank, YCSB and the sample_analytics dataset), the experiment harness and the insider attack suite. Out of scope are sharded clusters, prevention of tampering (the system detects, it does not block), a graphical interface and production operation; D.4.1 Section 1.4 lists the boundaries in full.

1.4 Link to the requirements

Table 1. Components and the requirements and user stories they implement

Component (module)	Functional requirements	User stories
Canonical hashing and Merkle trees (canonical.py, merkle.py, blocks.py)	FR02, FR07, FR18	US1, US6
Design S library and fold (design_s.py)	FR01 to FR03, FR07, FR08	US3
Design A service (design_a.py, mongo_store.py)	FR04 to FR08, FR10, FR11	US4, US5
Digest log and Git witness (digest_log.py)	FR08, FR09	US2
Verifier (verifier.py, scan.py)	FR12 to FR17	US1, US2
Search and proofs (search.py)	FR18 to FR20	US6
Command line (cli.py)	FR06, FR17 to FR20	US1, US5, US6
Workloads, harness, SQL Server baseline (workloads/, bench/, sqlserver/, ycsb/)	FR21 to FR24, FR27	US7
Attack suite and demonstration (attacks/, demo.py)	FR25, FR26	US8

2. Goals and Nongoals

Table 2. Design goals and how success is measured

Goal	Success criterion	Traces to
G1 Detect unexplained changes to protected data	Every attack in the D.4.1 threat table raises the expected finding	FR12 to FR16, AC03 to AC08
G2 Make ledger rewrites detectable once published	Rollback and consistent rewrite of a published block always raise an alert	FR13, AC06
G3 Leave MongoDB unmodified and free	Only public driver interfaces are used; all software is free	NFR09
G4 Keep the cost measurable and modest	Design A keeps at least 80 percent of baseline throughput at 16 clients; Design A lag at most 6 s at a 5 s interval	NFR04, NFR05
G5 Make audits practical	100,000 documents verified in under 60 s; any document's history and proof available in one command	NFR06, FR18 to FR20
G6 Make the work reproducible	Unit tests and CI pass; the full scenario runs from a fresh clone or a codespace	NFR11, NFR12

Nongoals. The system does not prevent an administrator from changing data; it records and exposes the change. It does not decide whether a recorded change was legitimate, which remains the auditor's judgment. It does not protect against an attacker who controls the database and every copy of the digest log at once. It does not support sharded clusters or provide a graphical interface, and it is not tuned for production throughput.

3. System Architecture

3.1 Logical architecture

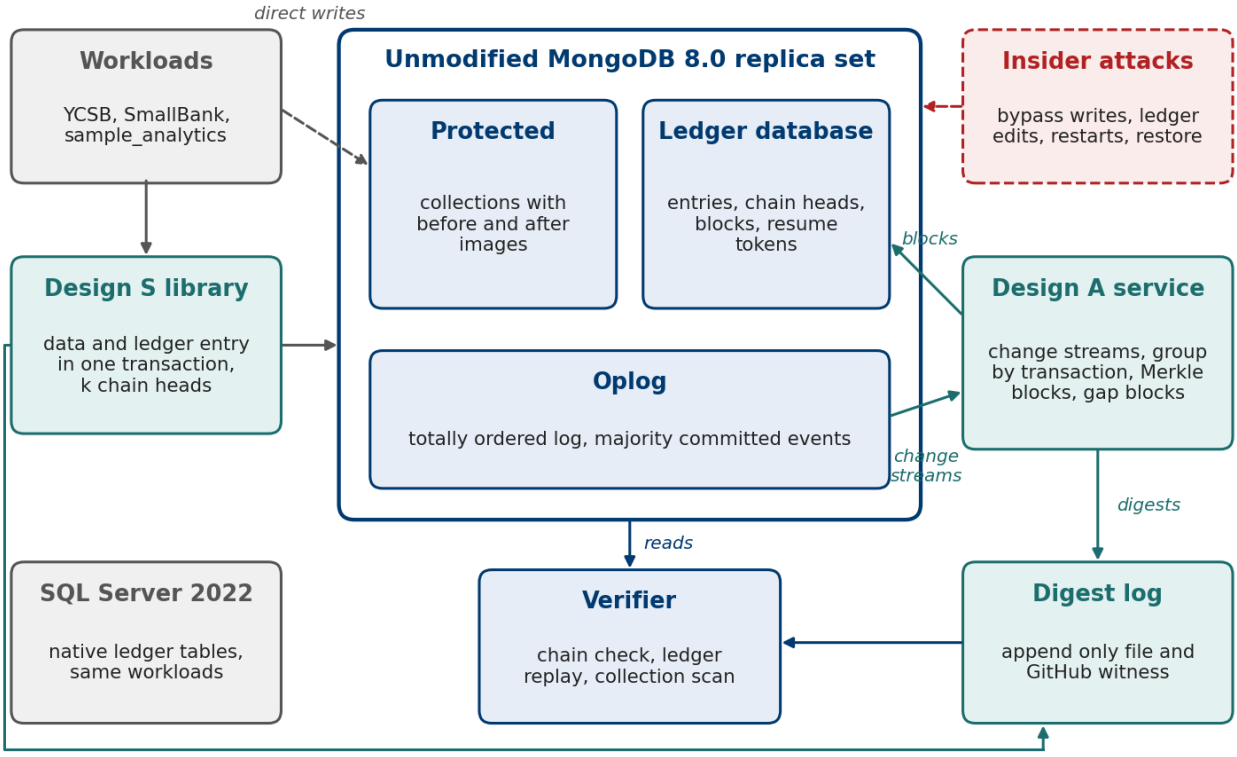


Figure 1. Logical architecture: two capture designs, one ledger format, one verifier

Figure 1 shows the subsystems and how they interact. **Workloads** write to the protected collections either directly or through the **Design S library**, which adds a ledger entry to each transaction and advances one of k chain heads. Independently, the **Design A service** reads the oplog through change streams, turns each committed change into an entry, groups the entries of one transaction, and seals Merkle blocks at a fixed interval. Both designs store blocks and entries in a **ledger database** on the same replica set and append each block's digest to the **digest log**, a file outside the database that can also be pushed to a separate Git repository as a witness. The **verifier** reads the ledger, the digests and a full scan of the data and reports findings; the search commands use the same stored ledger. **SQL Server 2022** runs the same workloads on its native ledger tables as the baseline, and the **attack suite** plays the insider.

Because both designs produce the same Entry and Block records, the verifier, the search commands and the experiment harness are shared. The two designs differ only in where entries are created, which is exactly the variable the project measures.

3.2 Physical architecture

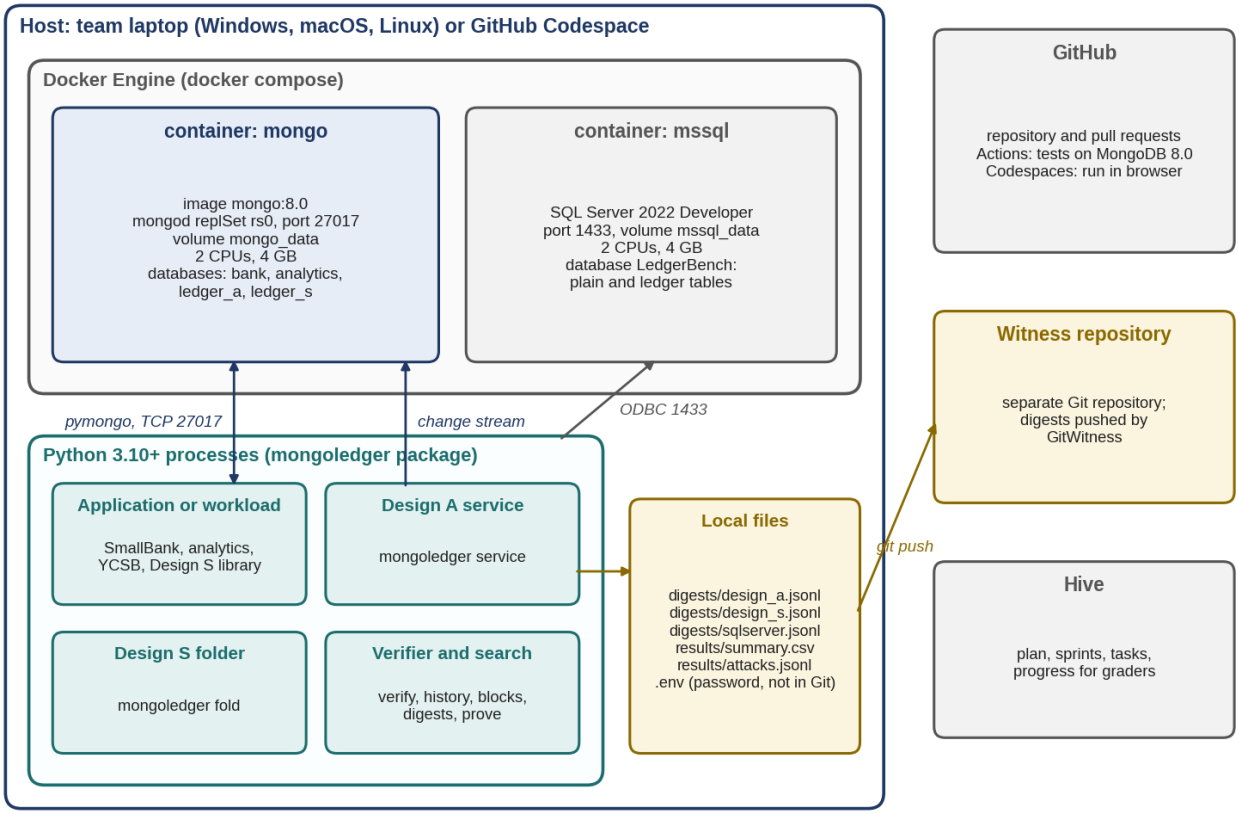


Figure 2. Physical architecture on a laptop or a GitHub Codespace

Figure 2 shows how the system is deployed. Docker Compose runs two containers on one host: **mongo** (MongoDB 8.0 started as the single member replica set rs0 on port 27017, with its data in the mongo_data volume) and **mssql** (SQL Server 2022 Developer edition on port 1433). Each container is limited to two CPUs and 4 GB so that runs are comparable, and only one database is measured at a time. The Python processes run on the host: the application or workload, the Design A service, the Design S fold loop and the verifier. They talk to MongoDB through pymongo over TCP and to SQL Server through ODBC Driver 18. The digest logs, the results and the .env file holding the SQL Server password stay on the host disk, outside both databases. GitHub hosts the code, runs the tests on a real MongoDB 8.0 container for every push (GitHub Actions) and provides a ready environment in the browser (Codespaces, defined in .devcontainer/); Hive holds the plan.

4. Data Model

4.1 Entity relationship model

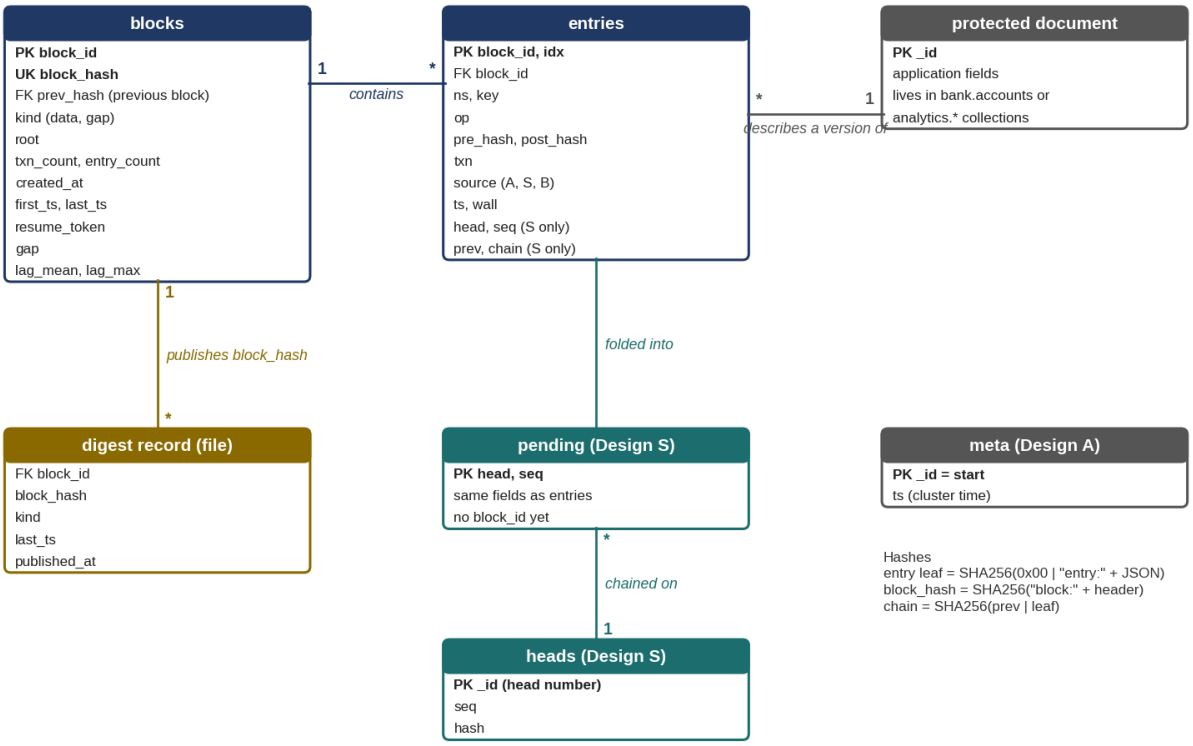


Figure 3. Ledger data model (MongoDB collections plus the digest file)

MongoDB enforces no schema, so the model in Figure 3 is logical; the repository documents it as $jsonSchema validators in schema/mongodb_schema.json. A block contains many entries, ordered by idx because the Merkle root depends on order; each block names the hash of its predecessor, which turns the blocks into a chain; and each entry describes one version of one protected document, identified by namespace and canonical key. In Design S, an entry first lives in pending, chained on one head, and is moved into a block by the fold step. A digest record in the external file publishes one block hash.

Table 3. Collections, keys and indexes

Collection	Key	Indexes	Written by
ledger_a.blocks, ledger_s.blocks	block_id	unique block_id	service, fold, setup
ledger_a.entries, ledger_s.entries	(block_id, idx)	unique (block_id, idx)	service, fold, setup
ledger_s.heads	_id = head number	primary key only	Design S library
ledger_s.pending	(head, seq)	unique (head, seq)	Design S library, fold
ledger_a.meta	_id = start	primary key only	setup
bank.accounts	_id = customer	primary key only	SmallBank
analytics.accounts, customers, transactions	_id (ObjectId)	account_id; customers.accounts; (account_id, transaction_count)	sample_analytics loader and workload

4.2 Hashes and canonical encoding

Every hash is SHA256 with a short prefix naming its kind, so a value of one kind cannot be passed off as another. A document hash is SHA256 of “doc:” followed by canonical Extended JSON of the document after a BSON round trip, with keys sorted at every level; the round trip truncates dates to milliseconds as the server does, and canonical mode keeps numeric type tags, so a hash taken from a change event equals a hash taken from a later scan. An

entry leaf is “entry:” followed by sorted JSON of the entry's core fields. A block hash covers every header field. Design S adds a chain link per head: chain equals SHA256 of the previous link followed by the leaf.

4.3 Block structure

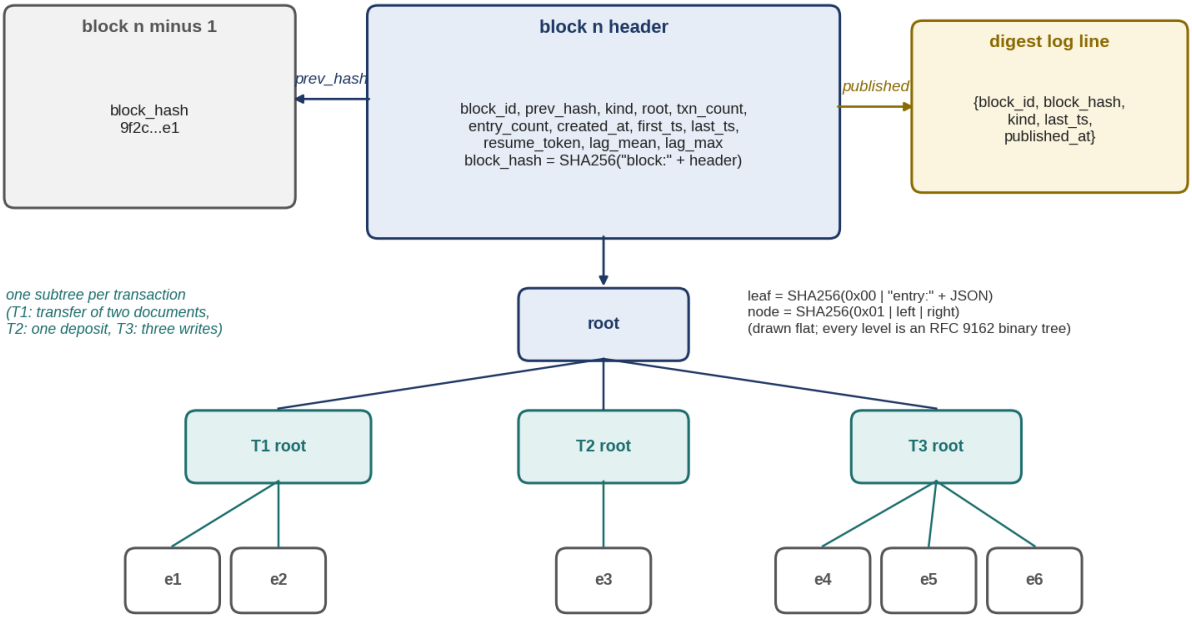


Figure 4. A block: header, two level Merkle tree and published digest

Figure 4 shows one block. Entries of the same transaction form a subtree whose root is a transaction root, and the block root is the RFC 9162 Merkle root over the transaction roots. Keeping a transaction in one subtree lets an auditor prove that both halves of a transfer were committed together, and makes every inclusion proof two short paths.

4.4 Data flow

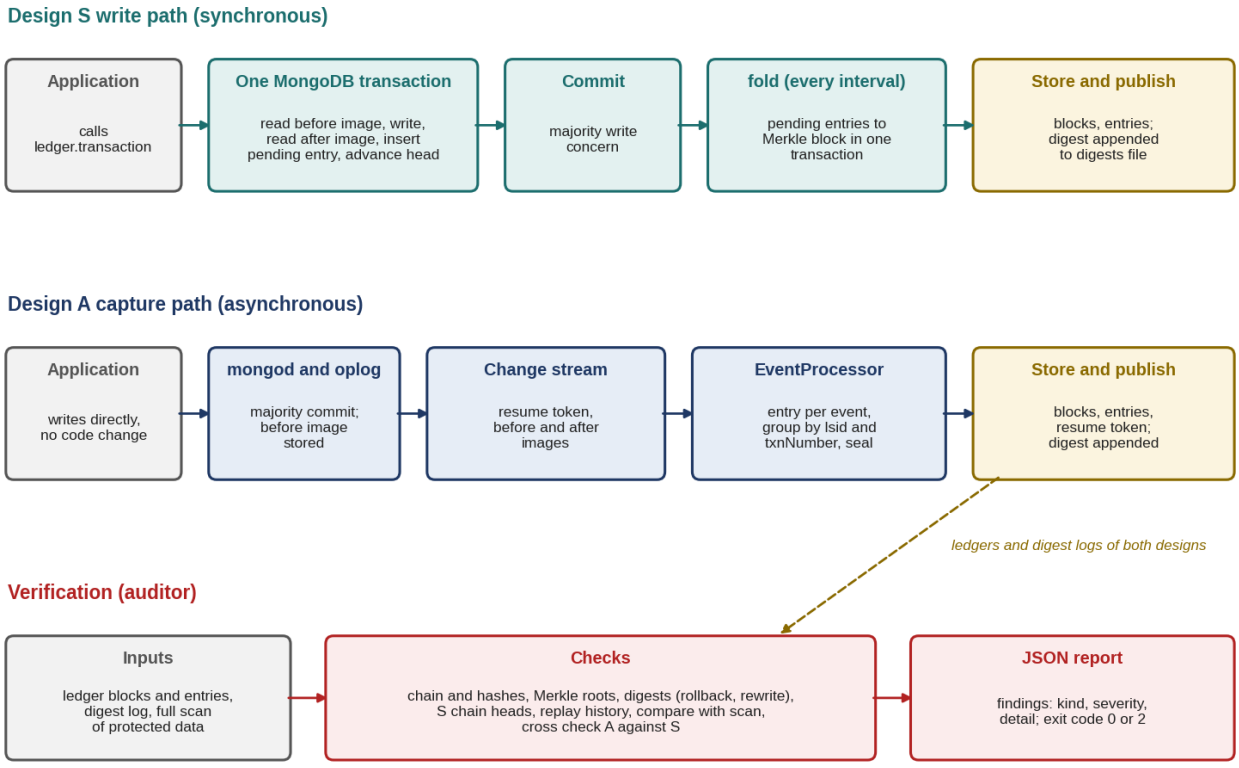


Figure 5. Data flow for the two capture paths and for verification

In Design S the ledger entry is created before the transaction commits, so the change and its evidence are atomic, and the fold step later seals committed entries. In Design A the application is untouched; the change reaches the ledger only after commit, through the change stream, which adds a delay (the commit to block lag) but also captures writes that bypass the application. Both paths end in the same store and the same digest log, which the verifier reads together with a full scan.

5. Component Design

This section gives the algorithms of the main modules as pseudocode. The Python source in the repository follows the same structure and carries extended comments on every module and function.

5.1 Merkle root and inclusion proof (merkle.py)

```
function MTH(items):                        # RFC 9162 section 2.1.1
    if items is empty: return SHA256("")
    if one item d:     return SHA256(0x00 | d)
    k = largest power of two smaller than len(items)
    return SHA256(0x01 | MTH(items[0:k]) | MTH(items[k:]))

function PATH(m, items):                    # audit path for leaf m
    if one item: return []
    k = largest power of two smaller than len(items)
    if m < k: return PATH(m, items[0:k]) + [MTH(items[k:])]
    else:     return PATH(m - k, items[k:]) + [MTH(items[0:k])]

verification recomputes the root from the leaf, its index, the tree size and the path
(RFC 9162 section 2.1.3.2) and accepts only if it equals the trusted root
```

5.2 Design S write and fold (design_s.py)

```
ledger.transaction(fn):
    with_transaction(read concern snapshot, write concern majority):   # retried on conflict
        t = new transaction context with id "s:" + random UUID
        fn(t)        # each t.update_one / insert_one / replace_one / delete_one does:
            before = find the document
            apply the write
            after  = find the document again                  # hash what the server stored
            h      = SHA256(namespace | key) mod k            # choose the chain head
            head   = read heads[h]                             # conflicts with concurrent writers
            entry  = {ns, key, op, hash(before), hash(after), txn,
                      seq = head.seq + 1, prev = head.hash}
            entry.chain = SHA256(entry.prev | leaf(entry))
            insert entry into pending
            set heads[h] = {seq, chain} only if heads[h].seq is unchanged

fold(store, digest_log):                       # every interval, in a separate process
    entries = pending ordered by (head, seq)
    block   = build_block(next id, previous block hash, entries, now)
    in one transaction: insert entries and block; delete them from pending
    digest_log.append(block.id, block.hash)    # fsync before returning
```

5.3 Design A service (design_a.py)

```
run_service(interval):
    position = last block's resume token, or the gap restart time, or the setup time
    loop:
        try:
            open change stream on the protected database at position,
                 fullDocument = whenAvailable, fullDocumentBeforeChange = whenAvailable
            for each event (or idle tick):
                entry = {ns, key, op, hash(before image) or UNAVAILABLE, hash(after image),
                         txn = "txn:" + hash(lsid) + txnNumber, or a unique id outside transactions,
                         ts = clusterTime, wall = wallTime}
                if interval elapsed:
                    seal pending entries, but if the stream is busy keep the trailing
                    entries of the last transaction for the next block (never split a transaction)
                    store block with the resume token of its last event; publish its digest
        on error 286 or 280 (resume point no longer in the oplog):
            seal what is pending
            seal a gap block {reason, last seen time, restart time}; publish it
            position = current cluster time
```

5.4 Verifier (verifier.py)

```
verify(blocks, entries, digests, scan, pending, heads):
    for each block in id order:
        alert block_sequence if ids are not contiguous from 0
        alert broken_link if prev_hash differs from the previous block hash
        alert block_hash_mismatch if the header no longer hashes to block_hash
        alert block_content_mismatch if the stored entries do not rebuild root
        warning gap for a gap block
    for each published digest:
        alert digest_without_block if its block is missing        # rollback or deletion
        alert digest_mismatch if the block hash differs          # rewrite after publication
    info unpublished_tail for blocks newer than the last digest
    Design S: check every head: seq contiguous, prev links, chain hashes, head matches newest entry
    replay entries in order, keeping expected[ns, key]:
        baseline sets the state; insert over an existing document is an alert;
        every other change must start from expected[ns, key], else alert history_break
    compare expected with the scan: missing_document, modified_document, unexpected_document
    optional cross check: every Design A change needs an unused identical Design S change,
        else alert write_outside_library
    report ok = no alert; exit code 0 or 2
```

5.5 Search and proofs (search.py)

The history command walks the sealed entries (and, for Design S, the pending ones) in ledger order and keeps those matching every filter given: document key, collection, operation, commit time window and block range. The blocks command summarizes each block, the digests command compares every published digest with the stored block and labels it ok, MISMATCH or MISSING BLOCK, and the prove command builds the two level inclusion

proof of Figure 4 for a chosen entry and checks it against the stored block root. All four are pure functions over the loaded ledger and are unit tested without a database.

6. Interface Design

The system has no graphical interface: auditors and operators use a command line, applications use a Python API, and the components exchange data through MongoDB and two text formats. The screen layouts below are the actual outputs of the commands, captured from the test environment; times and hashes differ on every run.

6.1 Command line interface

Table 4. Commands of the mongoledger program

Command	Main options	Output	Exit code
setup	`--heads k`	one line per design	0
service	`--interval s`, `--seconds n`	log lines with `-v`	0
fold	`--heads k`, `--every s`, `--seconds n`	none	0
verify	`--design A` or `S`, `--cross`	JSON report	0 clean, 2 alert
history	`--design`, `--key`, `--collection`, `--op`, `--since`, `--until`, `--from-block`, `--to-block`, `--limit`, `--full`, `--json`	table or JSON	0
blocks	`--design`, `--last n`, `--json`	table or JSON	0
digests	`--design`, `--json`	table or JSON	0 all ok, 2 otherwise
prove	`--design`, `--key`, or `--block` and `--idx`	JSON proof	0 verified, 2 failed, 1 not found

The same program runs as `python -m ledger.cli`. Global options come before the command: `--uri`, `--data-db`, `--collections` (comma separated), `--digest-log`, `--git-witness`, `-v`. MONGO_URI and DATA_DB set the defaults.

6.2 Screen layouts

Screen 1. Block listing (a two document transfer forms one transaction in each block)

```
$ mongoledger blocks --design A
block  kind  entries  txns  sealed                  lag_mean_s  lag_max_s  hash              note
=====  ====  =======  ====  ======================  ==========  =========  ================  ====
0      data  50       1     2026-10-21 16:24:33.383                        2569e656a3616a31
1      data  2        1     2026-10-21 16:24:33.402  0.001       0.001      d41702d6fc9874fd
2      data  2        1     2026-10-21 16:24:33.605  0.001       0.001      b1550df837f40401
```

Screen 2. History of one document

```
$ mongoledger history --design A --key 42
block  idx  time                     ns             key                          op        post_hash         source
=====  ===  =======================  =============  ===========================  ========  ================  ======
0      42                            bank.accounts  {"_id":{"$numberInt":"42"}}  baseline  8329943f1ef12a6a  B
1      0    2026-10-21 16:24:33.401  bank.accounts  {"_id":{"$numberInt":"42"}}  update    1375c7730905ab2d  A
2      0    2026-10-21 16:24:33.604  bank.accounts  {"_id":{"$numberInt":"42"}}  update    3e937502379adc5c  A
```

Screen 3. Verification report with an alert

```
$ mongoledger verify --design S                    (after an administrator edited account 42)
{
  "ok": false,
  "stats": {"blocks": 2, "entries": 56, "digests": 2, "documents_scanned": 50, ...},
  "findings": [
    {"kind": "modified_document", "severity": "alert",
     "detail": {"ns": "bank.accounts", "key": "{\"_id\":{\"$numberInt\":\"42\"}}"}}
  ]
}
$ echo $?
2
```

6.3 Python API

Table 5. Public Python interface

Call	Purpose
SyncLedger(client, data_db, ledger_db="ledger_s", heads=k)	Design S entry point; setup() creates the heads
ledger.transaction(fn)	Runs fn(t) atomically; t offers insert_one, update_one, replace_one, delete_one, find_one with a collection name first
ledger.fold(store, digest_log)	Seals pending entries into one block and publishes it; returns the block id
run_service(client, data_db, store, digest_log, interval)	Design A service loop
LedgerStore(client, ledger_db).load()	Returns (blocks, entries by block) in order
verify(blocks, entries, digests, scan, pending, heads, namespaces)	Returns a Report with findings, ok and stats
search.history / block_rows / digest_rows / prove	Pure functions behind the search commands

6.4 Protocols and file formats

- **MongoDB:** pymongo 4.8 or later over the MongoDB wire protocol, connection string mongodb://localhost:27017/?replicaSet=rs0&directConnection=true by default; majority write concern for ledger writes; change streams with fullDocument and fullDocumentBeforeChange set to whenAvailable.
- **SQL Server:** ODBC Driver 18; the baseline calls sp_generate_database_ledger_digest and sys.sp_verify_database_ledger with the stored digests.
- **Digest log:** JSON Lines, one object per block: block_id, block_hash, kind, last_ts and published_at; appended and flushed to disk with fsync.
- **Verification report:** JSON with ok, stats and findings, each finding holding kind, severity and detail.
- **Benchmark summary:** CSV with one row per run (workload, target, clients, heads, interval, throughput, p50, p95, p99, errors, lag, verification time, sizes).

7. Constraints and Security

7.1 Hardware and environment limits

All measurements run on one laptop, with each database container limited to two CPUs and 4 GB and only one database active at a time. The single node replica set is enough for transactions and change streams but cannot show replication lag or elections. SQL Server 2022 has no arm64 image, so on Apple Silicon it runs under Rosetta emulation or on another laptop, and emulated results are never compared with native ones. A GitHub Codespace offers two cores and 4 GB at minimum, enough for the demonstration but not for the final measurements.

7.2 Performance bottlenecks

- **Chain head contention (Design S):** transactions that choose the same head conflict and retry; the experiment with k of 1, 4 and 16 measures this cost.
- **Extra work per write (Design S):** two reads, one insert and one head update inside each transaction, plus hashing full documents, which grows with document size (sample_analytics buckets hold up to 100 trades).
- **Before images (Design A):** MongoDB stores a before image for each update in config.system.preimages, which costs write path I/O and storage until it expires.
- **Oplog window (Design A):** if the service is down longer than the oplog retains history, events are lost; the gap block makes the loss explicit and the scan still checks the final state.
- **Verification cost:** the verifier replays every entry and scans every protected document, so its time grows linearly with the ledger and the data.

7.3 Security considerations

- **Trust model:** the database administrator is not trusted; the evidence that survives is the digest log outside the database, ideally with a Git witness the team does not control. Blocks newer than the last published digest are protected only by the chain, and the verifier reports them as unpublished_tail.
- **Integrity primitives:** SHA256 with domain prefixes, RFC 9162 Merkle trees with distinct leaf and node prefixes, and canonical encoding, so equal documents hash equally and different values never collide by construction.
- **Consistency:** majority write concern and majority committed change events keep rolled back writes out of the ledger.
- **Credentials:** the SQL Server password lives only in .env, which Git ignores; .env.example holds a placeholder. The testbed MongoDB has no authentication; a deployment would enable TLS, create a service user with read access to the data and change streams plus write access to its ledger database only, and give auditors read only access.
- **Residual risk:** an attacker who controls the database and every digest copy can rewrite history undetected; detection of a change made through the library by a legitimate but malicious user is out of scope, because the ledger then records it faithfully.

8. Design Rationale

Table 6. Key decisions

Decision	Alternatives considered	Reason
Use change streams for Design A	Tailing local.oplog.rs directly; database triggers (not available in MongoDB Community)	Change streams are a supported, resumable interface that delivers only majority committed changes and, since 6.0, before and after images
Use multi document transactions for Design S	Writing entries after the data change; two phase commit in the application	Only a transaction makes the data change and its entry atomic, which is the defining property of Design S
k partitioned chain heads	One global chain; no chain (blocks only)	One chain serializes every writer; k heads spread contention while still detecting removed or reordered pending entries
RFC 9162 Merkle trees with SHA256	Plain hash chain of entries; a custom tree	Logarithmic inclusion proofs, a published standard that independent tools can check, and protection against leaf and node confusion
Transaction subtrees inside blocks	One flat tree of entries	A transaction can be proven as a unit and is never split across blocks
Canonical Extended JSON after a BSON round trip	Hashing raw BSON bytes; plain JSON	Field order and date precision differ between sources; canonical mode keeps numeric types exact

Decision	Alternatives considered	Reason
JSON Lines digest log with an optional Git witness	Azure confidential ledger; a blockchain	Free, simple and append only; a Git host gives an independent, timestamped history
Python with pymongo	Java or Node.js	Short, readable code for the team, first class driver support and fast tests
SQL Server 2022 Ledger as baseline	PostgreSQL extensions; Amazon QLDB (retired)	A native engine ledger the course already uses, free in Developer edition
Docker Compose and Codespaces	Local installation; MongoDB Atlas	Identical environments on Windows, macOS and Linux; graders can run the system without installing anything
In memory stand in for unit tests	Requiring a server for every test	Fast tests on every push; integration tests still run on a real MongoDB 8.0 in CI

9. Revision History

Version	Date	Author	Change	Approved by
0.1	28 Oct 2026	Project team	First draft from the implemented system and D.4.1 version 1.0	Pending
1.0	30 Oct 2026	Project team	Reviewed by all three members and submitted as D.4.2	O.O, R.N, Y.P

References

[1] O. Ogunjebi, R. Nelluri, and Y. Pratap, “User requirements and analysis document,” COMP.8157 Deliverable D.4.1, University of Windsor, Oct. 2026.

[2] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, IETF, Dec. 2021.

[3] MongoDB, Inc., “Change streams,” MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/changeStreams/

[4] MongoDB, Inc., “Transactions,” MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[5] W. Schultz, T. Avitabile, and A. Cabral, “Tunable consistency in MongoDB,” Proc. VLDB Endow., vol. 12, no. 12, 2019.

[6] P. Antonopoulos et al., “SQL Ledger: Cryptographically verifiable data in Azure SQL Database,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2021.

[7] R. T. Snodgrass, S. S. Yao, and C. Collberg, “Tamper detection in audit logs,” in Proc. 30th Int. Conf. Very Large Data Bases (VLDB), 2004.